How Our Robot Answers Business Questions

A family field guide to the Retail SQL Agent

Written for two of my favorite people: a curious 10-year-old, and a sharp business mind.

You don't need to know how to code to understand this book — just curiosity.

Chapter 1 — What Is This Thing, Really?

Somewhere on Dad's computer lives a little robot. You type it a question in plain English — like "What were our top 5 products by revenue?" — and a few seconds later it types back a short, honest answer, using real numbers from a real database of store sales.

That's the whole trick. It doesn't guess. It doesn't make things up. It goes and looks, every single time, and it tells you exactly what it found — including when it isn't sure about something.

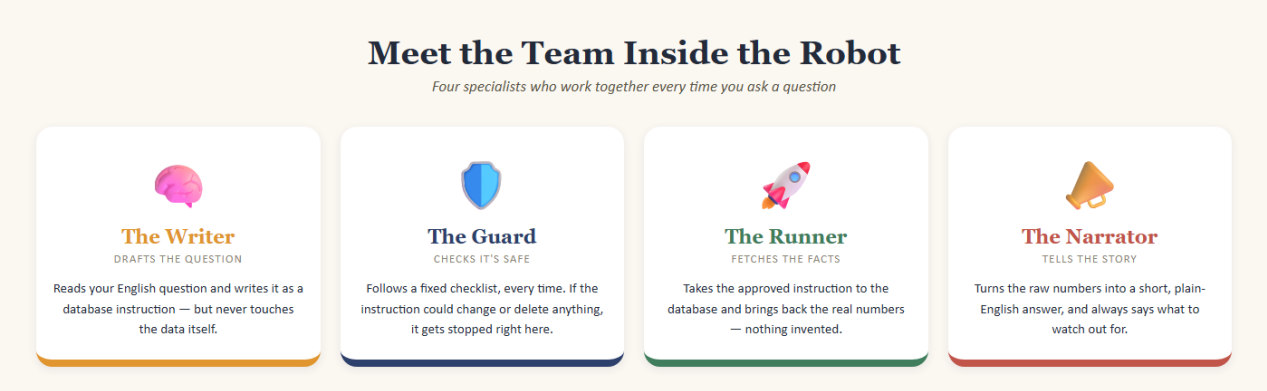
> **KID'S EYE VIEW**
> Think of a robot librarian. You ask it a question, it walks into a giant filing cabinet full of receipts, pulls out exactly the right ones, counts them up, and comes back to tell you the answer. It promises never to rip up a receipt, never to reorganize the cabinet, and never to lie about what it found.

> **BUSINESS LENS**
> This is a natural-language interface over a SQL database. Normally, answering "which store had the most returns this month" means finding someone who knows SQL, waiting for them to write and run a query, and hoping they interpreted the question correctly. This agent collapses that round-trip into a single typed sentence — with a safety layer that guarantees it can only read data, never change it.

It was built as a learning project — a chance to understand, hands-on, how modern "AI agents" are actually put together under the hood, rather than treating them as magic.

Chapter 2 — Meet the Team Inside the Robot

It's tempting to imagine one big brain doing everything. In reality, this robot is more like a small, well-organized office. Four specialists, each with exactly one job, hand work to each other in a fixed order — every single time, for every single question.



The four specialists, and the one job each of them is allowed to do.

KID'S EYE VIEW

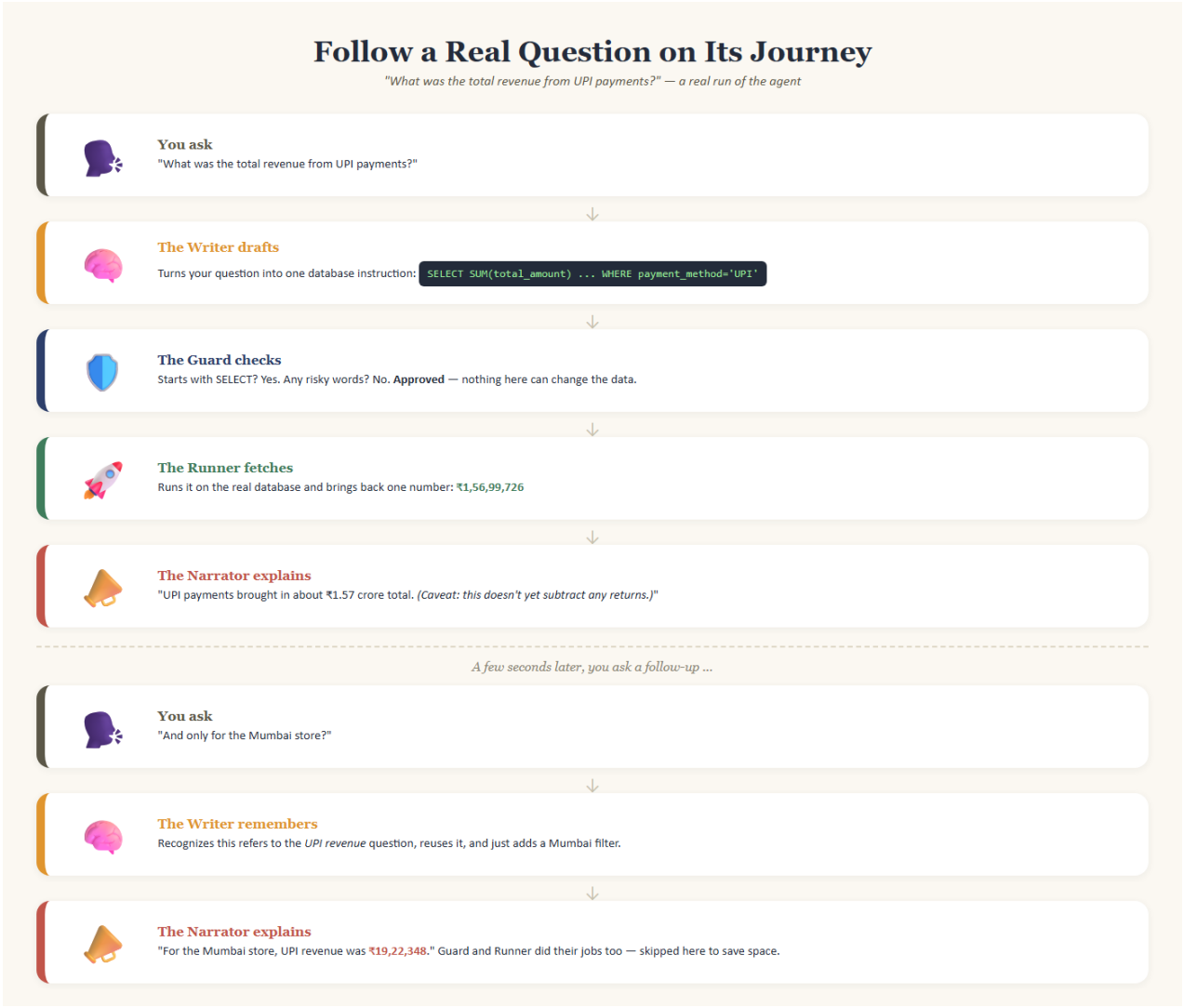
It's a bit like a relay race. The Writer runs the first leg and hands off the baton to the Guard, who hands off to the Runner, who hands off to the Narrator. Nobody skips their leg, and nobody runs someone else's.

BUSINESS LENS

This is a deliberate design choice called separation of concerns. One AI model drafts the query; a completely separate, non-AI piece of code approves or rejects it; a database driver executes it; a second AI call turns the result into prose. No single component can both decide something is safe and act on it — the same principle behind requiring two signatures on a large payment.

Chapter 3 — Follow a Real Question on Its Journey

Here's an actual run of the robot, end to end, including a natural follow-up question — the kind of thing you'd really ask in a conversation.



A real question and a real follow-up, exactly as the agent handled them.

Notice what happened with the follow-up. You didn't have to repeat yourself and say "what was the UPI revenue for the Mumbai store" — you just said "and only for the Mumbai store?" and the Writer understood you meant the question right before it. That's conversation memory: the robot is handed the last few turns of your chat every time, with the most recent one clearly flagged, so it never has to guess which question a follow-up belongs to.

A BUG WE ACTUALLY CAUGHT

Early on, when there were several unrelated questions earlier in the conversation, the Writer sometimes reused the wrong one — it would grab an older question's structure instead of the one right before the follow-up. We caught this by independently checking the numbers by hand, fixed it by explicitly labeling which turn was "most recent," and re-tested to confirm it was fixed. That's the human review step in action — never trust an AI's output for something that matters without checking it.

Chapter 4 — The Rules That Keep It Safe

The single most important design decision in this whole project is about the Guard. It would have been easier to just tell the AI, in the prompt, "please only write safe SELECT queries and never delete anything." That's tempting — but it's not good enough, because an AI model can occasionally misunderstand, get tricked, or simply have an off moment.

So the Guard isn't AI at all. It's plain, boring, predictable code — the kind that does the exact same thing every time, forever. It checks, in order:

1. Is this exactly one instruction? (Not five instructions hidden in one message.)
2. Does it start with the word SELECT? (SELECT only reads data — it can never change or delete anything.)
3. Does it contain any of 12 forbidden words — things like DELETE, DROP, UPDATE, or ALTER — anywhere in it?

If any check fails, the question never reaches the database at all. The robot just replies that it can't do that — politely, and without pretending to have tried.

> **KID'S EYE VIEW**
> It's like airport security. The scanner doesn't sit there and think about whether your bag looks trustworthy today — it runs the exact same scan on everyone, every time. That's actually what makes it safe: no favorites, no bad days, no talking your way past it.

> **BUSINESS LENS**
> A rule-based control is auditable in a way an AI "promise" is not — you can point to the code, write a test for every rule, and prove it holds 100% of the time. That's the difference between a control you can put in front of a compliance reviewer and one you can only hope works. In this project it's called "defense-in-depth": even though the AI is also told to only write safe queries, the deterministic Guard is what actually enforces it.

Chapter 5 — When Things Go Wrong

Sometimes the Writer's first attempt at a query is a little off — maybe it uses a feature the database doesn't actually support. Rather than giving up immediately, the robot tries a small, bounded recovery: it takes the exact error message the database gave back, hands it to the Writer along with the original question, and asks for a corrected attempt. This can happen up to twice before the robot gives up honestly and says so.

A REAL EXAMPLE

One database feature (putting a "top N" limit inside a certain kind of sub-question) simply isn't supported by MySQL, the database this project uses. The first attempt failed with a clear error. That error was fed straight back to the Writer, which rewrote the query a different way — and it worked on the second try. No human had to step in while it was running; the system healed itself using the database's own error message.

BUSINESS LENS

This bounded retry (capped at two attempts, always) is what makes the system resilient without becoming unpredictable. A system that retries forever can hang or loop up costs; a system with zero retries fails on every small hiccup. Capping it, and failing honestly afterward, is a deliberately small, cheap failure mode.

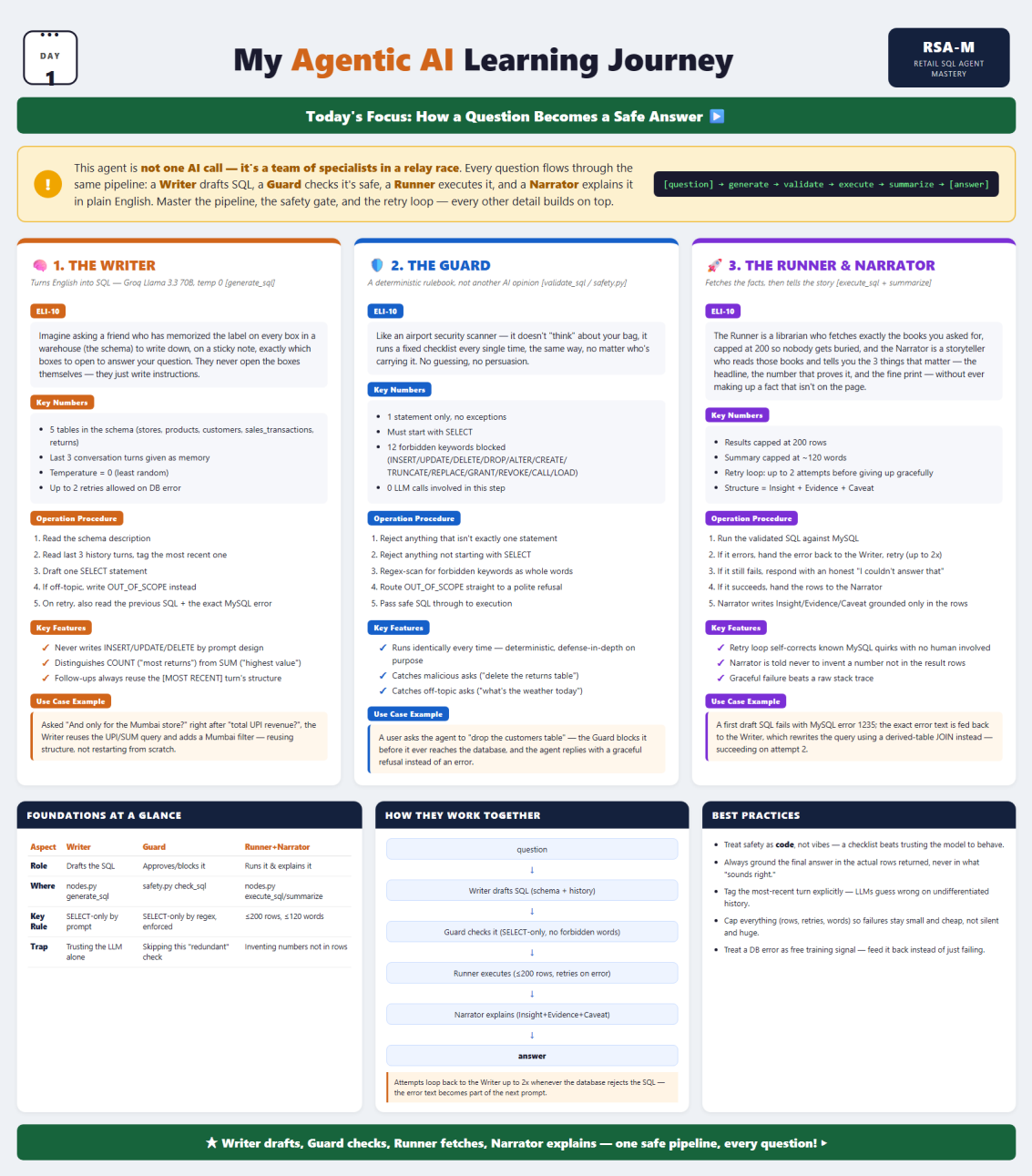
Chapter 6 — New Words You'll Hear

A short glossary, in plain language, for the vocabulary that comes up when talking about this project.

AI / LLM ("Large Language Model")	A computer program trained on enormous amounts of text, so it can read a question and write a reasonable-sounding response. It's very good at language, but it doesn't actually "know" facts the way a database does — which is exactly why this project never lets it answer from memory alone.
Prompt	The instructions and information you hand the AI right before asking it to respond — in this project, that includes a description of the database and your recent conversation.
Temperature	A setting that controls how "creative" vs. "predictable" the AI's answers are. This project uses the lowest setting (0), because for writing database queries, boring and consistent beats creative.
SQL	"Structured Query Language" — the standard language used to ask a database questions like "how many" or "what's the total." SELECT is the SQL word for "just show me," as opposed to words that change data.
Database / Table	A very organized set of spreadsheets (called tables) that a computer can search through instantly. This project's database has five tables: stores, products, customers, sales transactions, and returns.
Schema	A description of what each table contains and how the tables relate to each other — like a table of contents for the database, so the Writer knows what questions are even answerable.
API Key	A private password-like code that proves you're allowed to use an AI service. It's kept in a hidden settings file and is never shared, printed, or saved anywhere public — treat it exactly like a bank PIN.
Agent / LangGraph	"Agent" is the general term for a program that uses AI to take a sequence of actions toward a goal, instead of just answering one question. LangGraph is the specific tool used here to wire the Writer, Guard, Runner, and Narrator together into that fixed relay-race order.

Chapter 7 — The Full Technical Map

For anyone who wants the fuller picture — the exact numbers, the file names, the precise rules — here's the one-page reference version of everything in this book. You don't need to memorize it. Just know it's here to come back to.



The full technical reference poster for this project's pipeline.

Chapter 8 — Build It Yourself: A Weekend Project

This whole robot runs on Dad's laptop — no special hardware, and the AI service it uses is free. Here's how to bring it up from nothing and have your own conversation with it. Budget about 30–45 minutes the first time, with a grown-up nearby to help read error messages.

> **BEFORE YOU START**
> This is exactly what real software engineers do: follow a checklist, one step at a time, and don't panic if something doesn't work on the first try. It almost never does, for anyone.

Step 1 — Gather the ingredients

Three things need to be installed once: Python (the programming language the robot is written in), uv (a tool that installs all the smaller pieces the project needs), and MySQL (the database itself).

Step 2 — Get a library card for the AI

Sign up for a free account at Groq (the company providing the AI "brain") and copy your personal API key — a long code that's your private ticket to use it.

Step 3 — Give the robot its secrets, privately

Copy the file named .env.example to a new file named .env, and paste in your API key and your database password. This file is like a locked diary: the robot reads it to log in, but it's never shown to anyone else and never saved on the internet.

Step 4 — Fill the pretend store with data

Run one command (uv run scripts/load_data.py) and the robot will build its five tables and fill them with a year's worth of made-up — but realistic — store sales, so there's something real to ask about.

Step 5 — Say hello

Run uv run app.py, and you'll get a prompt where you can just start typing questions in plain English. Try Chapter 9 for ideas.

> **PREFER A WEB PAGE?**
> There's also a browser version, if typing into a plain black window isn't your thing. Run uv run streamlit run app_streamlit.py instead, and it opens a proper chat page — the same robot, the same rules, just with a friendlier face and a table for the numbers.

IF SOMETHING BREAKS

Copy the exact red error message and read the last line first — it almost always says what went wrong in plain words ("file not found," "access denied," "connection refused"). Every programmer, including the one who built this, sees these messages daily. They're not a sign anything is broken forever — just a clue to fix.

Chapter 9 — Questions to Try Asking

For the curious 10-year-old

- How many stores do we have?
- What's the most popular way people pay — cash, card, or UPI?
- Which product category sells the most?
- How many products do we sell in total?
- What's the most expensive product in the store?

For the business mind

- What are our top 5 products by revenue?
- Which store has the highest return rate?
- How much revenue came through UPI versus credit card?
- Which city's stores are generating the most sales?
- What percentage of revenue is lost to returns?

> **TRY THIS TOO**
> Ask it something the database genuinely can't answer — like "what will the weather be tomorrow?" — and watch it politely decline instead of making something up. Then try asking it to "delete all the returns" and watch the Guard step in. Both are working exactly as designed.

Chapter 10 — Quiz Time!

No peeking at the answer key until you've had a guess. Play together — some of these are just as fun for grown-ups.

1. Which team member checks whether a question is safe before it ever reaches the database?

- The Writer
- The Guard
- The Runner
- The Narrator

Answer: The Guard

2. True or False: The Narrator is allowed to make up a number if it isn't sure of the real answer.

- True
- False

Answer: False — it must only use numbers that actually came back from the database.

3. Put these in the correct order: Runner fetches · Guard checks · Narrator explains · Writer drafts.

Answer: Writer drafts → Guard checks → Runner fetches → Narrator explains

4. If a question's SQL contains the word DROP, what happens?

- It runs anyway, carefully
- It gets blocked and the robot politely declines
- It asks a grown-up for permission first

Answer: It gets blocked and the robot politely declines

5. Discuss: why use a fixed checklist for safety instead of just asking the AI to "please be careful"?

Answer: Open discussion — a checklist runs the same way every time and can be proven correct; an AI's promise can occasionally be misunderstood or tricked.

6. Bonus for the business mind: name one reason a company would insist the safety check be deterministic code rather than another AI's judgment call.

Answer: Open discussion — auditability: you can test and prove a rule holds 100% of the time, which you can't do with an AI's judgment.

Chapter 11 — Where to Go Further

Everything in this book has a "real" version in the actual project folder, for whenever you're ready to look closer:

- README.md — the full setup and architecture notes
- ai_usage.md — every bug that was caught during human review, and how it was fixed
- rubric.md — how the finished project was evaluated against its original brief

None of this required being a professional programmer to understand at the level this book covers — just the willingness to ask "but why?" one layer at a time. That's the whole skill. Go ask the robot something.